

Species Distribution Modelling of Lantana camara in India

Author

Rishabh Tiwari

Hibah Patanwala

B.Sc. Applied Mathematical Computing

Research Discourse II, Semester 5

Supervisor

Jyoti Prajapati

October 21, 2026

Abstract

Biological invasions impose severe ecological and socio-economic costs, making the accurate prediction of their potential distribution and identification of suitable environmental covariates that facilitate their growth critical for proactive management. *Lantana camara* is a highly noxious invasive shrub in India, yet mapping its suitable habitat is complicated by the complex, highly correlated nature of environmental data. This study models the potential distribution of L. camara across India while systematically evaluating the efficacy of different dimensionality reduction techniques and machine learning algorithms. Using occurrence records paired with environmentally stratified background points, the model integrates 19 bioclimatic variables, 3 anthropogenic variables and 2 hydrological variables. To resolve multicollinearity, we compared correlation based variable selection and Variance Inflation Factor (VIF) against Principal Component Analysis (PCA) . The resulting datasets were utilized to train, optimize, and evaluate a comprehensive suite of machine learning classifiers: Random Forest, Decision Tree, K-Nearest Neighbors (KNN), Logistic Regression, XGBoost, and Support Vector Machines (SVM). Comparative evaluation of multiple machine learning frameworks revealed that a Variance Inflation Factor (VIF) feature selection pipeline paired with a tuned Random Forest classifier produced the optimal Species Distribution Model for Lantana camara, achieving a highly accurate final ROC AUC of 0.9082 on the isolated spatial test set. Feature importance analysis identified anthropogenic disturbance—specifically proximity to urban areas, railways, and roads—along with seasonal climate extremes such as precipitation of the coldest quarter, as the primary environmental determinants driving *Lantana camara* invasion.

Table of Contents

1 Introduction & Literature Review **4**

2 Methodology **5**
2.1 Data Curation & Preparation 5
2.1.1 Occurrence Data 5
2.1.2 Environmental Covariates 5
2.1.3 Background Points 7
2.2 Variable Selection and Dimensionality Reduction 7
2.3 Machine Learning Classifiers 8

3 Results **10**
3.1 Variable Selection and Dimensionality Reduction Outcomes 10
3.2 Monte Carlo Simulation based Model Comparison 10
3.3 Model Evaluation 11
3.4 Interpretation 12

4 Appendix **13**

References **20**

1 Introduction & Literature Review

Biological invasions represent one of the most severe drivers of global biodiversity loss and ecosystem degradation. *Lantana camara*, a perennial flowering shrub native to the tropical regions of the Americas, is globally recognized as one of the world's most aggressive invasive alien species. Originally introduced to various parts of the world as an ornamental plant, it has since escaped cultivation and naturalized across tropical, subtropical, and warm temperate environments. The species possesses highly adaptable physiological traits, including prolific seed production, rapid vegetative growth, and allelopathic properties that actively inhibit the germination and growth of surrounding native flora. By forming dense, impenetrable thickets, *L. camara* severely alters local ecological dynamics, disrupting native plant lifecyles, degrading wildlife habitats, and altering soil chemistry.

In India, Lantana camara was introduced in the early 19th century and has since become pervasive across a vast array of the country's landscapes, from the Western Ghats to the foothills of the Himalayas. Its unchecked proliferation poses a massive threat to both conservation areas and agricultural lands. Recent studies estimate the scale of invasion in India to be greater than 300,000 sq. km. (>40%) of total forest cover. The shrub competes directly with native floral species, severely impacting grazing capacities for both livestock and native herbivores, which in turn causes cascading economic disruptions for local communities reliant on these ecosystems. Given the sheer scale of its spread, eradicating established populations of *L. camara* is often financially and logistically prohibitive, making early detection and proactive spatial management the most viable conservation strategies.

Species Distribution Modeling (SDM) is a mathematical framework used to predict the geographic range of a species by establishing empirical relationships between known occurrence records and spatially explicit environmental covariates. The fundamental workflow involves aggregating geo-referenced species presence data—often supplemented by systematically generated pseudo-absences or background points—and extracting corresponding values from multi-dimensional raster layers representing climatic, topographic, and anthropogenic predictors. Because environmental variables often exhibit severe spatial autocorrelation and multicollinearity, the predictor space is typically refined using feature selection techniques. These refined inputs are then processed through advanced statistical or machine learning algorithms to learn the complex, non-linear ecological thresholds governing species survival. The final mathematical model is projected back onto geographic space to generate a continuous predictive surface map, where each pixel is assigned a habitat suitability index or probability of presence, allowing researchers to identify fundamental ecological niches, potential invasion zones, and conservation priorities.

Species distribution modeling (SDM) in India plays a crucial role in understanding biodiversity patterns and informing conservation strategies. MaxEnt (Maximum Entropy) algorithm has emerged as the preferred tool for SDM in India, with a notable focus on plants and mammals (Ramanan et al., 2025). A notable study by Deomurari et al. (2023) developed SDMS for 1091 bird species, producing detailed distribution maps covering 75% of Indian avifauna. Most SDM studies have concentrated on native species distribution, with less emphasis on invasive species(Roy et al., 2022). Kishore et al. (2023) used species distribution modelling to map invasive potential of Chromolaena odorata and Lantana camara across India's Western Ghats, relating bioclimatic and topographical variables to current habitats, then projecting hotspots under SSP future climates; results identified major present and changing invasion areas.

The canonical reference for our study is Iyer, Bannerjee & Bhowmick (2019). In their study

they model the presence of invasive plant *Mikania micrantha* using Logistic regression based on climatic variables selected on the basis of two cross regularization methods induced by least absolute shrinkage and selection operator (lasso) and ridge penalty function. We build upon the paper by including anthropogenic and hydrological variables and a range of Machine Learning classifiers (discussed below) in addition to Logistic regression.

2 Methodology

2.1 Data Curation & Preparation

2.1.1 Occurrence Data

Global species occurrence data was downloaded from the Global Biodiversity Information Facility (GBIF) using the `occ` function in `pygbif` module in Python. The records were obtained in the form of a csv file which was imported to Python and stored as a **Geodataframe** using the `geopandas` library in Python. Using spatial operations facilitated by Geopandas, Indian occurence dataset was created. To resolve spatial autocorrelation, only one record was retained from each site. Utilizing this pipeline, 3464 occurrences were obtained within the boundaries of India. The data collection process in Python is outlined in Appendix.

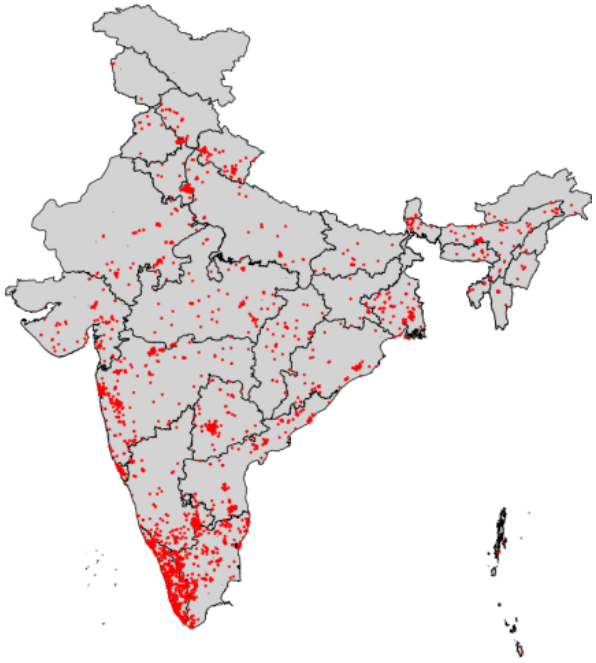


Figure 1: *Lantana camara* occurences in India

2.1.2 Environmental Covariates

In our study we have considered a total of 24 independent variables:

Climatic variables

- BIO1: Annual Mean Temperature
- BIO2: Mean Diurnal Range (Mean of monthly Max Temperature − Min Temperature)

- BIO3: Isothermality $(\frac{\text{BIO2}}{\text{BIO7}} \times 100)$
- BIO4: Temperature Seasonality (Standard Deviation $\times 100$)
- BIO5: Max Temperature of Warmest Month
- BIO6: Min Temperature of Coldest Month
- BIO7: Temperature Annual Range $(\text{BIO5} - \text{BIO6})$
- BIO8: Mean Temperature of Wettest Quarter BIO9: Mean Temperature of Driest Quarter
- BIO10: Mean Temperature of Warmest Quarter
- BIO11: Mean Temperature of Coldest Quarter
- BIO12: Annual Precipitation
- BIO13: Precipitation of Wettest Month
- BIO14: Precipitation of Driest Month
- BIO15: Precipitation Seasonality (Coefficient of Variation)
- BIO16: Precipitation of Wettest Quarter
- BIO17: Precipitation of Driest Quarter
- BIO18: Precipitation of Warmest Quarter
- BIO19: Precipitation of Coldest Quarter

Hydrological variables

- Distance to nearest river (km)
- Distance to nearest reservoir/lake (km)

Anthropogenic variables

- Distance to nearest railway line (km)
- Distance to nearest road (km)
- Distance to nearest urban settlement (km)

The hydrological and anthropogenic variables were downloaded as **shapefiles (SHP)** and distances to occurrence and background points were calculated by converting the shapefiles to Geodataframes and applying `sjoin_nearest` method in Geopandas. The climatic variables were installed as **TIFF** files and extracted at both occurrence and background points using spatial point querying implemented in `rasterio` module in Python.

2.1.3 Background Points

In Species Distribution Modeling, predictive algorithms require both presence data (labeled as 1) and background or pseudo-absence data (labeled as 0) to establish decision boundaries. To generate an environmentally rigorous set of background points, we employed a stratified sampling technique based on the Köppen-Geiger climate classification system.
The **Köppen-Geiger** system is a widely used empirical framework that categorizes global climates based on annual and monthly averages of temperature and precipitation. Stratifying background points by these climate zones is essential for building a robust correlative model. If background points were purely randomly sampled across the entirety of India's diverse landscapes, the model might simply learn to differentiate broad, obvious macro-climates (e.g., tropical versus arid regions). By forcing the background points to strictly mirror the broad climatic distribution of the species occurrence data, we compel the machine learning classifiers to learn on the basis of finer variation in the values of the predictive features listed above.
The stratification was computationally executed in Python via a multi-step pipeline. First, the specific Köppen-Geiger climate zone was extracted for each of the known *Lantana camara* occurrence points. Next, a large pool of random geographic coordinates was generated across the Indian landmass, and their corresponding climate zones were extracted. Finally, pseudo-absence points were sampled from this random pool in exact proportion to the climate zone frequencies observed in the occurrence dataset. We include as many background points as the occurrence points (3464). The implementation is contained in the Appendix.

2.2 Variable Selection and Dimensionality Reduction

Species Distribution Models (SDMs) leveraging high-dimensional environmental datasets — such as the 24 combined climatic, hydrological, and anthropogenic variables utilized in this study— frequently suffer from severe multicollinearity. Bioclimatic variables (e.g., annual mean temperature and maximum temperature of the warmest month) often exhibit strong linear dependencies. Left unaddressed, multicollinearity inflates the variance of coefficient estimates, reduces the statistical power of the model, and leads to severe overfitting, which degrades the model's ability to generalize to novel geographic regions. To isolate the most predictive environmental signals while maintaining model stability, three distinct approaches were evaluated: Correlation-based feature selection, Variance Inflation Factor (VIF) filtering, and Principal Component Analysis (PCA).

Correlation-Based Feature Selection
Correlation analysis serves as a primary filter to identify and eliminate highly collinear variables before modeling. The linear relationship between any two continuous environmental variables, X and Y, is quantified using the Pearson correlation coefficient (r):

$$r = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^{n}(X_i - \bar{X})^2 \sum_{i=1}^{n}(Y_i - \bar{Y})^2}}$$

A predefined collinearity threshold (typically $|r| \geq 0.5$) is established. For any pair of variables exceeding this threshold, the variable with the lesser ecological relevance is removed. This directly mitigates multicollinearity while preserving the original units and interpretability of the retained variables.

Variance Inflation Factor (VIF)
VIF is a rigorous variable selection technique that quantifies the extent to which the variance

of an estimated regression coefficient is increased due to collinearity with other predictors. For each predictor variable X_i, an ordinary least squares regression is performed against all other predictor variables in the dataset. The VIF for X_i is calculated as:

$$VIF_i = \frac{1}{1 - R_i^2}$$

where R_i^2 is the coefficient of determination from the regression of X_i on the remaining covariates. A threshold is iteratively applied (commonly $VIF \leq 5$ or 10), and variables exceeding this limit are sequentially removed.

Principal Component Analysis (PCA)
Unlike VIF and correlation analysis, PCA is a dimensionality reduction technique that fundamentally transforms the feature space rather than discarding original variables. PCA applies an orthogonal linear transformation to the standardized dataset, projecting the highly correlated covariates into a new set of linearly uncorrelated variables called principal components. Given a mean-centered data matrix X, PCA computes the covariance matrix Σ:

$$\Sigma = \frac{1}{n-1} X^T X$$

Eigendecomposition is then performed on Σ to yield eigenvalues λ and eigenvectors v. The eigenvectors represent the directions of the new feature space, while the eigenvalues dictate the magnitude of variance captured along each component. Components are selected based on the cumulative explained variance (e.g., 90%). While PCA highly optimizes predictive efficiency, it obscures direct feature interpretability, as each principal component is a linear combination of all original variables.

2.3 Machine Learning Classifiers

Following preprocessing and dimensionality reduction, a comprehensive suite of machine learning algorithms was deployed to predict the spatial presence of *Lantana camara*. These classifiers represent a diverse spectrum of parametric, instance-based, and ensemble learning paradigms.

Logistic Regression (LR)
Logistic Regression serves as the foundational parametric baseline for binary classification. It models the probability that a given geographic coordinate supports *Lantana camara* presence by fitting the log-odds as a linear combination of the environmental predictors. The probability $P(y = 1|X)$ is computed via the logistic sigmoid function:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta^T X)}}$$

where β_0 is the intercept and β represents the vector of learned coefficients.

K-Nearest Neighbors (KNN)
KNN is a non-parametric, instance-based classifier that assigns a class label to a novel spatial coordinate based on the majority vote of its k closest data points in the training set. Proximity is typically quantified using the Euclidean distance metric in the n-dimensional environmental feature space:

$$d(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x_i')^2}$$

Decision Tree (DT)
The Decision Tree algorithm recursively partitions the environmental feature space into distinct, non-overlapping regions. At each internal node, the algorithm selects the feature and threshold that best split the data into pure classes. The optimal split is determined by minimizing an impurity metric, such as Gini Impurity:

$$I_G(p) = \sum_{i=1}^{C} p_i(1 - p_i)$$

where C is the number of classes and p_i is the probability of an item belonging to class i.

Random Forest (RF)
Random Forest is a robust ensemble learning method that mitigates the overfitting tendencies of individual decision trees through bootstrap aggregating (bagging) and random feature selection. The algorithmic construction proceeds as follows for an ensemble of B trees:

1. Draw a bootstrap sample (with replacement) of size N from the training dataset.
2. Grow a decision tree $h_b(x)$ on this sampled data. At each node, rather than evaluating all predictors, select a random subset of m features (typically $m \approx \sqrt{p}$, where p is the total number of predictors).
3. Determine the optimal split solely from this subset of m features.

For a novel spatial observation x, the final spatial prediction is determined by aggregating the independent predictions of all B trees via majority voting:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \ldots, h_B(x)\}$$

This stochastic construction ensures that the individual trees are decorrelated, significantly reducing the variance of the final model and providing stable predictions across complex environmental gradients.

Extreme Gradient Boosting (XGBoost)
XGBoost is an advanced ensemble technique that builds decision trees sequentially. Each new tree is specifically optimized to correct the residual errors made by the combination of all previous trees. XGBoost minimizes a regularized objective function $\mathcal{L}$:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

where l is a differentiable convex loss function measuring the difference between the target y_i and prediction $\hat{y}_i$, and Ω penalizes the complexity of the trees.

Support Vector Machines (SVM)
SVM constructs a hyperplane in a high-dimensional feature space to optimally separate presence and background data points. The algorithm identifies the decision boundary that maximizes the margin—the distance between the hyperplane and the nearest training data points (support vectors). To construct this optimal boundary, SVM solves the following dual optimization problem:

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

subject to the constraints $\sum_{i=1}^{N} \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$, where C is a regularization parameter controlling the penalty for misclassification, α_i are Lagrange multipliers, and $y_i \in \{-1, 1\}$ represent the binary class labels.

Because species distributions rarely exhibit strictly linear relationships with environmental gradients, we deployed SVM with a Radial Basis Function (RBF) kernel. The RBF kernel implicitly maps the environmental covariates into an infinite-dimensional space to handle complex, non-linear physiological thresholds:

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

where γ defines the influence radius of a single training example on the decision boundary.

3 Results

3.1 Variable Selection and Dimensionality Reduction Outcomes

To mitigate severe multicollinearity among the initial 24 environmental and anthropogenic predictors, the dataset was independently processed through three feature-handling pipelines prior to model training: Correlation-based selection, Variance Inflation Factor (VIF) filtering, and Principal Component Analysis (PCA).

- **Correlation Filtering:** An absolute correlation threshold of 0.5 was applied, reducing the feature space to 8 independent predictors. The explicitly retained variables were: distance to railways, distance to roads, distance to urban settlements, distance to lakes, Annual Mean Temperature (BIO1), Annual Precipitation (BIO12), Precipitation of the Driest Month (BIO14), and Precipitation of the Warmest Quarter (BIO18).
- **Variance Inflation Factor (VIF):** Variables were sequentially removed if they exceeded a collinearity threshold of $VIF \geq 5$. This process eliminated 11 highly collinear bioclimatic variables (such as Temperature Annual Range (BIO7), which exhibited extreme initial variance inflation). The 13 retained predictors were: distance to railways, distance to roads, distance to urban areas, distance to lakes, and distance to rivers; Precipitation of the Wettest Month (BIO13), Driest Month (BIO14), Warmest Quarter (BIO18), and Coldest Quarter (BIO19); Mean Diurnal Range (BIO2); Isothermality (BIO3); and Mean Temperature of the Wettest Quarter (BIO8) and Driest Quarter (BIO9).
- **Principal Component Analysis (PCA):** Rather than selecting discrete variables, PCA transformed the standardized dataset into a set of orthogonal principal components. The algorithm dynamically retained the precise number of principal components required to capture $\geq 90\%$ of the cumulative environmental variance from the original 24 variables. While this pipeline completely eliminated multicollinearity and maximized variance retention, it inherently obscured the direct ecological interpretability of the individual input features.

3.2 Monte Carlo Simulation based Model Comparison

To robustly identify the optimal modeling framework without bias from a single train-test split, a Monte Carlo simulation was executed across 3 independent iterations. Each iteration generated unique random spatial splits and model seeds to evaluate six machine learning classifiers across the three distinct feature-handling pipelines.

	Pipeline	Accuracy	Precision	Recall	F1	ROC_AUC
Decision Tree	corr	0.7512	0.7853	0.6921	0.7355	0.8191
KNN	corr	0.8108	0.7909	0.8451	0.8171	0.8937
Logistic	corr	0.7371	0.7187	0.7790	0.7476	0.8113
Random Forest	corr	0.8321	0.8413	0.8188	0.8299	0.9057
SVM	corr	0.7901	0.8034	0.7684	0.7855	0.8578
XGBoost	corr	0.8232	0.8253	0.8201	0.8226	0.8948
Decision Tree	vif	0.7595	0.7675	0.7466	0.7564	0.8255
KNN	vif	0.8058	0.7857	0.8412	0.8125	0.8921
Logistic	vif	0.7342	0.7141	0.7809	0.7460	0.8081
Random Forest	vif	0.8326	0.8361	0.8275	0.8317	0.9061
SVM	vif	0.7956	0.8029	0.7838	0.7932	0.8642
XGBoost	vif	0.8222	0.8242	0.8191	0.8216	0.8962
Decision Tree	pca	0.7597	0.7669	0.7460	0.7560	0.8130
KNN	pca	0.8137	0.8014	0.8342	0.8174	0.8963
Logistic	pca	0.7447	0.7299	0.7771	0.7527	0.8183
Random Forest	pca	0.8230	0.8332	0.8076	0.8202	0.9015
SVM	pca	0.8034	0.8081	0.7960	0.8020	0.8696
XGBoost	pca	0.8095	0.8159	0.7992	0.8075	0.8890

Figure 2: Model performance under different pipelines

Ensemble tree-based architectures consistently outperformed linear and distance-based classifiers across all data pipelines. The Random Forest algorithm, paired specifically with the VIF feature selection pipeline, emerged as the superior modeling framework. It achieved the highest mean performance metrics across all iterations:

- **Mean ROC AUC:** 0.9061
- **Mean Accuracy:** 0.8326
- **Mean F1-Score:** 0.8317
- **Mean Precision:** 0.8361
- **Mean Recall:** 0.8275

XGBoost under the VIF pipeline was the second-most effective model, yielding a mean ROC AUC of 0.8962. Conversely, Logistic Regression consistently exhibited the lowest predictive capability across all configurations, indicating that the ecological thresholds determining *Lantana camara* presence are highly non-linear. Models trained on VIF-reduced datasets generally outperformed those trained on PCA-transformed data, suggesting that preserving precise, non-linear relationships between individual covariates is more predictive than the composite variance maximized by PCA.

3.3 Model Evaluation

Given its superior performance in the Monte Carlo simulation, the VIF + Random Forest configuration was selected as the definitive Species Distribution Model. To finalize the model architecture, hyperparameters were optimized via 5-fold GridSearchCV, yielding an optimal tree structure of `n_estimators`: 200, `min_samples_split`: 2, and an unconstrained `max_depth` (None).
Evaluated on the isolated 30% spatial test set, the tuned model exhibited exceptional predictive power:

- Accuracy: 0.8345

- Precision: 0.8390
- Recall: 0.8277
- F1-Score: 0.8333
- ROC AUC: 0.9082

To rigorously test resilience to spatial autocorrelation, a spatial block cross-validation utilizing a 2.0-degree geographic grid was executed. This spatial cross-validation returned a mean AUC of 0.825 (± 0.021), compared to the standard random-split cross-validation AUC of 0.914 (± 0.006). This expected performance drop confirms that standard random splits can provide optimistic metrics; however, an AUC of 0.825 validates that the model fundamentally maintains robust predictive generalization when forecasting into novel, spatially independent regions.

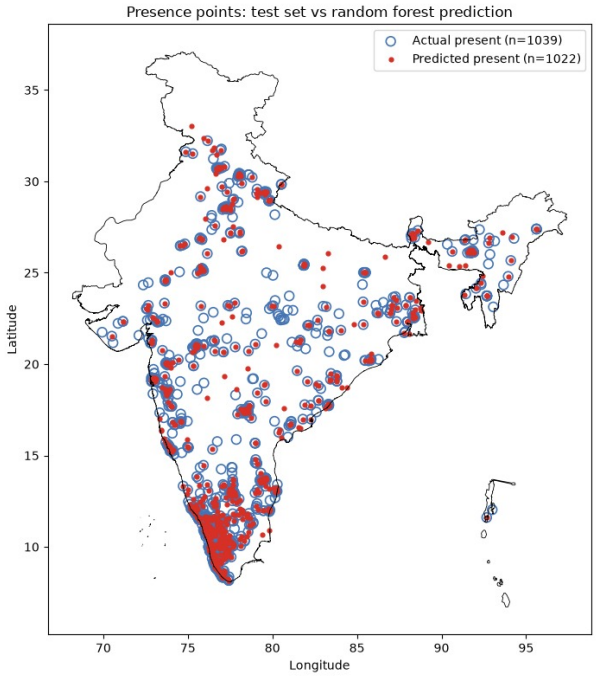


Figure 3: Model predictions

3.4 Interpretation

Extracting the Gini impurity-based feature importances from the final Random Forest model provided direct insights into the primary environmental drivers of *Lantana camara* distribution. The model heavily prioritized anthropogenic disturbance metrics alongside specific seasonal temperature and precipitation variables. The most influential predictors, in descending order of importance, were:

1. Distance to urban areas
2. Distance to railways
3. Precipitation of the coldest quarter (BIO19)
4. Distance to roads

5. Isothermality (BIO3)

The dominance of proximity to urban settlements, railways, and roads strongly indicates that *L. camara* invasion is highly opportunistic, leveraging human-disturbed landscapes and transportation corridors for rapid establishment and dispersal.

4 Appendix

Obtaining Indian occurences of *Lantana camara*:

```
from pygbif import occurrences as occ
from pygbif import species
import geopandas as gpd

taxon_LC=species.name_backbone(scientificName='Lantana
 ↪  camara')
taxon_key=taxon_LC['usage']['key']
print(taxon_key)

occurrence_records=occ.search(
    taxonKey=taxon_key,
    hasCoordinate=True,
    hasGeospatialIssue=False
    )

occurrence_records_all=occ.download(
    queries=[f'taxonKey = {taxon_key}',
    'hasCoordinate = True',
    'hasGeospatialIssue = False'],
    user = '********',
    pwd = '********',
    email = '********')

gdf_actual = gpd.GeoDataFrame(
    df_actual,
    geometry =
     ↪  gpd.points_from_xy(df_actual['decimalLongitude'],
     ↪  df_actual['decimalLatitude']),
    crs = 'EPSG:4326'
    )

india_map=gpd.read_file('C:/Users/johnn/.spyder-py3/India_Stat
 ↪  e_Boundary(www.simplygis.in)/India_State_Boundary.shp')
gdf_india = gpd.clip(gdf_actual, india_map)
```

Extraction of anthropogenic and hydrological features:

```
import geopandas as gpd
import pandas as pd
```

```
indian_rail = gpd.read_file('C:/Users/johnn/.spyder-py3/INDIAN
 ↪  _RAILWAYS.geojson')
ir_mtr = indian_rail.to_crs('EPSG:7755')
occ_rail = gpd.sjoin_nearest(
    gdf_india_mtr,
    ir_mtr,
    how='left',
    distance_col='dist_in_meters'
    )
occ_rail['dist_in_km_railways'] =
 ↪  occ_rail['dist_in_meters']/1000

indian_road = gpd.read_file('C:/Users/johnn/.spyder-py3/India
 ↪  Shapefile/IND_rds/IND_roads.shp')
ird_mtr = indian_road.to_crs('EPSG:7755')
occ_road = gpd.sjoin_nearest(
    gdf_india_mtr,
    ird_mtr,
    how='left',
    distance_col='dist_in_meters'
    )
occ_road['dist_in_km_road'] = occ_road['dist_in_meters']/1000

indian_river = gpd.read_file('C:/Users/johnn/.spyder-py3/India
 ↪  Shapefile/IND_wat/IND_water_lines_dcw.shp')
irv_mtr = indian_river.to_crs('EPSG:7755')
occ_river = gpd.sjoin_nearest(
    gdf_india_mtr,
    irv_mtr,
    how='left',
    distance_col='dist_in_meters'
    )
occ_river['dist_in_km_river'] =
 ↪  occ_river['dist_in_meters']/1000

indian_cities = gpd.read_file('C:/Users/johnn/.spyder-py3/citi
 ↪  es-&-towns-india.geojson')
irc_mtr = indian_cities.to_crs('EPSG:7755')
occ_urban = gpd.sjoin_nearest(
    gdf_india_mtr,
    irc_mtr,
    how='left',
    distance_col='dist_in_mtr'
    )
occ_urban['distance_in_km_urban'] =
 ↪  occ_urban['dist_in_mtr']/1000
```

```
lakes = gpd.read_file('C:/Users/johnn/.spyder-py3/HydroLAKES_p
↪  olys_v10_shp/HydroLAKES_polys_v10_shp/HydroLAKES_polys_v10
↪  .shp')
india_lakes = gpd.clip(lakes, india_map)
irl_mtr = india_lakes.to_crs('EPSG:7755')
occ_lakes = gpd.sjoin_nearest(
    gdf_india_mtr,
    irl_mtr,
    how='left',
    distance_col='dist_in_mtr'
    )
occ_lakes['distance_in_km_lakes'] =
↪  occ_lakes['dist_in_mtr']/1000

merge_keys = ['decimalLatitude', 'decimalLongitude']
occ_rail = occ_rail.drop_duplicates(subset=merge_keys)
occ_road = occ_road.drop_duplicates(subset=merge_keys)
occ_river = occ_river.drop_duplicates(subset=merge_keys)
occ_urban = occ_urban.drop_duplicates(subset=merge_keys)
occ_lakes = occ_lakes.drop_duplicates(subset=merge_keys)

factor_df = occ_rail.copy()
factor_df = factor_df.merge(
    occ_road[merge_keys + ['dist_in_km_road']],
    on=merge_keys,
    how='inner'
)
factor_df = factor_df.merge(
    occ_river[merge_keys + ['dist_in_km_river']],
    on=merge_keys,
    how='inner'
)
factor_df = factor_df.merge(
    occ_urban[merge_keys + ['distance_in_km_urban']],
    on=merge_keys,
    how='inner'
)
factor_df = factor_df.merge(
    occ_lakes[merge_keys + ['distance_in_km_lakes']],
    on=merge_keys,
    how='inner'
)

columns_to_drop = ['dist_in_meters', 'dist_in_mtr']
factor_df = factor_df.drop(columns=columns_to_drop,
↪  errors='ignore')

factor_df2 = pd.read_csv('C:/Users/johnn/.spyder-py3/India_Lan
↪  tana_19_Bioclim_Values.csv')
```

```
factor_df2 = factor_df2.drop_duplicates(subset =
 ↪  ['longitude','latitude'])
factor_df2 = factor_df2.rename(columns={
    'longitude': 'decimalLongitude',
    'latitude': 'decimalLatitude'
})

consolidated_df = pd.merge(factor_df, factor_df2, how='inner',
 ↪  on = ['decimalLatitude','decimalLongitude'] )
```

Extraction of climate rasters:

```
import pandas as pd
import os
import glob
import rasterio

df = pd.read_csv("lantana_full_dataset.csv")
coords = [(x, y) for x, y in zip(df['longitude'],
 ↪  df['latitude'])]

tif_folder = r"C:\Users\hibah\rd\image"
tif_files = glob.glob(os.path.join(tif_folder, "*.tif"))

for tif_path in tif_files:
    col_name = os.path.basename(tif_path).replace('.tif', '')

    with rasterio.open(tif_path) as src:
        extracted_values = [val[0] for val in
         ↪  src.sample(coords)]
        df[col_name] = extracted_values
```

Background generation:

```
import numpy as np
import pandas as pd
import geopandas as gpd
import rasterio
from shapely.geometry import Point

target_points = len(lantana_camara_india)
zone_percentages = lantana_camara_india['koppen_zone'].value_c
 ↪  ounts(normalize=True)
points_per_zone = (zone_percentages *
 ↪  target_points).round().astype(int)

minx, miny, maxx, maxy = india_map.total_bounds
np.random.seed(42)
x_rand = np.random.uniform(minx, maxx, 500000)
y_rand = np.random.uniform(miny, maxy, 500000)
```

```
candidate_gdf = gpd.GeoDataFrame(
    geometry=[Point(x, y) for x, y in zip(x_rand, y_rand)],
    crs="EPSG:4326"
)

candidate_gdf = gpd.sjoin(candidate_gdf, india_map,
 ↪  how='inner', predicate='intersects')
candidate_gdf = candidate_gdf.drop_duplicates(subset=['geometr
 ↪  y']).reset_index(drop=True)

coords_bg = [(x, y) for x, y in zip(candidate_gdf.geometry.x,
 ↪  candidate_gdf.geometry.y)]
with rasterio.open('koppen_geiger_0p00833333.tif') as src:
    candidate_gdf['koppen_zone'] = [val[0] for val in
     ↪  src.sample(coords_bg)]

candidate_gdf = candidate_gdf[candidate_gdf['koppen_zone'] !=
 ↪  0]

stratified_background = pd.DataFrame()

for zone, count in points_per_zone.items():
    if count == 0:
        continue
    zone_candidates =
     ↪  candidate_gdf[candidate_gdf['koppen_zone'] == zone]
    safe_count = min(count, len(zone_candidates))
    sampled_points = zone_candidates.sample(n=safe_count,
     ↪  random_state=42)
    stratified_background = pd.concat([stratified_background,
     ↪  sampled_points])

stratified_background = stratified_background.sample(frac=1,
 ↪  random_state=42).reset_index(drop=True)
stratified_background =
 ↪  stratified_background.head(target_points)

stratified_background['decimalLongitude'] =
 ↪  stratified_background.geometry.x
stratified_background['decimalLatitude'] =
 ↪  stratified_background.geometry.y
stratified_background['presence'] = 0
```

Comparison of model performance under different variable selection/dimensionality reduction techniques:

```
def drop_highly_correlated(data, threshold=0.5):
    data = data.copy()
```

```
    while True:
        corr = data.corr().abs()
        upper = corr.where(np.triu(np.ones(corr.shape),
         ↪  k=1).astype(bool))
        max_corr = upper.max().max()
        if max_corr < threshold:
            break
        data = data.drop(columns=[upper.max().idxmax()])
    return data

corr_features = drop_highly_correlated(X_all,
 ↪  CORR_THRESHOLD).columns.tolist()

def calculate_vif(data):
    Xc = add_constant(data)
    return pd.DataFrame({
        'Feature': data.columns,
        'VIF': [variance_inflation_factor(Xc.values, i + 1)
         ↪  for i in range(data.shape[1])]
    })

X_vif = df[predictor_cols].copy()
while X_vif.shape[1] > 1:
    vif = calculate_vif(X_vif)
    max_vif = vif['VIF'].max()
    if max_vif < 5:
        break
    drop = vif.loc[vif['VIF'].idxmax(), 'Feature']
    X_vif = X_vif.drop(columns=[drop])

records = []

for i in range(0, N_ITER):
    seed = i
    print(f'\n=== Iteration {i + 1}/{N_ITER} (seed={seed})
     ↪  ===')

    for pipe in PIPELINES:
        if pipe == 'corr':
            X = X_all[corr_features]
        elif pipe == 'vif':
            X = X_all[vif_features]
        else:
            X = X_all

        X_train, X_test, y_train, y_test = train_test_split(
            X, y_all, test_size=0.3, random_state=seed,
             ↪  stratify=y_all
        )
```

```
        scaler = StandardScaler()
        X_train_s = scaler.fit_transform(X_train)
        X_test_s = scaler.transform(X_test)

        n_comp = np.nan

        if pipe == 'pca':
            pca_full = PCA().fit(X_train_s)
            n_comp = int(np.argmax(np.cumsum(pca_full.explaine
             ↪  d_variance_ratio_) >= 0.90) + 1)

            pca = PCA(n_components=n_comp)
            X_train_f = pca.fit_transform(X_train_s)
            X_test_f = pca.transform(X_test_s)
        else:
            X_train_f, X_test_f = X_train_s, X_test_s

        for name, (est, grid) in get_models(seed).items():
            if TUNE_EACH_ITER or FIXED_PARAMS[pipe][name] is
             ↪  None:
                gs = GridSearchCV(est, grid, cv=5,
                 ↪  scoring='roc_auc', n_jobs=-1)
                gs.fit(X_train_f, y_train)
                best, best_params = gs.best_estimator_,
                 ↪  gs.best_params_
            else:
                best_params = FIXED_PARAMS[pipe][name]
                best = est.set_params(**best_params).fit(X_tra
                 ↪  in_f, y_train)

            row = {
                'Pipeline': pipe,
                'Iteration': i,
                'Seed': seed,
                'Model': name,
                'n_components': n_comp,
                'Best_Params': str(best_params)
            }
            row.update(evaluate(best, X_test_f, y_test))
            records.append(row)

            print(f" [{pipe}] {name:14s}
             ↪  AUC={row['ROC_AUC']:.4f}
             ↪  Acc={row['Accuracy']:.4f}")
```

Optimizing the Random Forest Algorithm:

```
rf_params = {
    'n_estimators': [100, 200, 500],
```

```
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10]
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    rf_params,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1
)

rf_grid.fit(X_train_scaled, y_train)
rf_best = rf_grid.best_estimator_
print(rf_grid.best_params_)

y_pred = rf_best.predict(X_test_scaled)
y_prob = rf_best.predict_proba(X_test_scaled)[:, 1]

results = pd.Series({
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1': f1_score(y_test, y_pred),
    'ROC_AUC': roc_auc_score(y_test, y_prob),
}).round(4)

print(results)
```

References

[1] Iyer, Bannerjee & Bhowmick. (2017). *Making choices that matter - Use of statistical regularization in species distribution modelling for identification of climatic indicators - A case study with Mikania micrantha Kunth in India*. Ecological Indicators, Elsevier

[2] https://www.gbif.org/

[3] https://worldclim.org/

[4] https://www.quickmaptools.com/

[5] https://www.gloh2o.org/koppen/

[6] https://docs.geopandas.org/en/latest/docs.html

[7] https://rasterio.readthedocs.io/en/latest/index.html

[8] https://scikit-learn.org/stable/user_guide.html